

#ASLI ENGINEERING

String Internals

SWIPE

BY

ARPIT BHAYANI

String Internals

Strings in Redis are implemented using SDS (Simple Dynamic String)

Why own implementation ?

- accessing string length is efficient
- appends are efficient
- binary safe ie. it can have NULL in b/w

There are 5 types of layout

sdshdr5

- unsigned char flags 3 lsb for type, 5 msb for length
- char buf[]

sdshdr8

- uint8 len length of the string
- uint8 alloc total size of the buffer allocated
- unsigned char flags 3 lsb for type, 5 msb unused
- char buf[]

Similarly there are sdshdr16, sdshdr32 and sdshdr64

We can store all kinds of data in SDS,

The 3 encodings are RAW, EMBSTR, INT

ARPIT BHAYANI

EMBSTR v/s RAW

EMBSTR is string whose length <= 44

embedded string is stored in the Redis Object itself

```
struct redisObject {
    unsigned type: 4;          }
    unsigned encoding: 4;      } 1B
    unsigned lru: LRU_BITS;    } 3B
    int refcount;              } 4B
    void * ptr;                } 8B
```

sdshdr8

```
    - uint8 len                } 1B
    - uint8 alloc              } 1B
    - unsigned char flags      } 1B
    - char buf[]               0B
}
```

$64 - 16 - 3 = 45$

↓

$44 + \backslash 0$

Hence EMBSTR has limit of 44 bytes

Flexible array members occupies 0 bytes. and it holds the leftover.

INT encoding : if str is representable as int, it converts and holds it in * ptr

* During implementation, we stored it as string and that is wrong!

ARPIT BHAYANI